ANTIGRAVITY MASTER AGENT SETUP

Version 2 - cleaned, consolidated, copy-ready

This document is an instruction specification for the coding agent. It does not claim that any repository, package, skill, MCP server or service is already installed. The agent must inspect, verify, install and test each item before reporting success.

What changed from v1

- Docs reduced from 14 to 10 files: icon rules, component-reuse rules and dependency policy are now sections inside `ENGINEERING_RULES.md`.
- One single skill registry (`docs/engineering/SKILL_REGISTRY.md`). The v1 duplicate under `.agents/` is removed.
- The two open items (Animaster source, task-chunking repository) now have fill-in slots, so you can complete them without editing the prompt body.
- The master prompt is deduplicated and ordered by gates. Same safety rules, less repetition, fewer tokens.
- Nothing new was added to scope: no extra tools, services or integrations.

1. Objective and architecture

Configure the existing repository as a durable, modular, graph-powered Antigravity environment. The agent loads only relevant skills and code context, reuses existing architecture, makes the smallest complete change, and verifies it. This is a setup task, not a redesign or rewrite.

Layer	Responsibility	Holds
Agent Skills	Task-specific expertise, loaded on demand	SKILL.md, references, scripts, templates
Code Knowledge Graph	Code structure and relationships	Files, symbols, imports, callers/callees, components, routes, tests, impact
Onyx (optional)	Searchable knowledge / RAG platform	Authorized docs, indexed sources, search, optional MCP

Keep the three separate. A full application repository is not an Agent Skill. Onyx is a platform, not a folder with a SKILL.md. A graph never replaces reading real source before a risky edit.

2. Requested integrations

Treat these as ten requested capabilities. For each, the agent determines whether upstream is a native Agent Skill, CLI, library, service, or a mix. URLs below are intended targets, not guarantees.

#	Name	Candidate source	Purpose
1	REA	github.com/morluto/rea	Authorized reverse engineering
2	Ponytail	github.com/DietrichGebert/ponytail	Minimal, clean implementation
3	Hallmark	github.com/Nutlope/hallmark	UI/design capability (verify identity)
4	Humanizer	github.com/hannsxpeter/humanizer	Natural-language polish
5	Onyx	github.com/onyx-dot-app/onyx	Knowledge/RAG platform + MCP. NOT a skill
6	Impeccable	github.com/pbakaus/impeccable	UI audit and refinement
7	UI UX Pro Max	github.com/nextlevelbuilder/ui-ux-pro-max-skill	Design-system / UI-UX intelligence

#	Name	Candidate source	Purpose
8	Last30Days	github.com/mvanhorn/last30days-skill	Recent research
9	Scrapling	github.com/D4Vinci/Scrapling	Scraping toolkit (Python package vs skill)
10	Diagram Design	github.com/cathrynlavery/diagram-design	Technical diagrams

If a target is missing, renamed, archived, ambiguous or incompatible, the agent stops that item and reports it. It never substitutes another repository.

2.1 UI libraries

Library	Source	Rule
Animaster	NOT CONFIRMED	Do not guess or install a similar name. PENDING USER INPUT.
Skiper UI	skiper-ui.com/docs/quick-start	Follow current quick-start; component-level install only.
Vengence UI	vengenceui.com	Verify exact spelling, registry and component command first.

2.2 Open items you can fill in

Item	Status	Your input
Animaster official URL / package	PENDING USER INPUT	URL: _____
Task-chunking skill repository	PENDING USER INPUT	URL: _____

Until these are supplied, task chunking runs as a local workflow (Phase 8) and no third-party task skill is installed.

3. Token-efficient skill routing

Task intent	Load
Reverse engineer authorized runtime/app behavior	REA
Refactor, simplify or implement code	Ponytail + relevant graph context
Create or redesign a UI surface	UI UX Pro Max and/or Hallmark; Impeccable for audit
Polish natural-language copy	Humanizer
Recent technology/community research	Last30Days
Authorized website extraction	Scrapling
Architecture / data / sequence diagrams	Diagram Design
Search project knowledge base	Onyx MCP, only if deployed and connected
Onyx setup/operations	Official Onyx docs, not a coding skill

- Never activate all skills for every request. Read metadata first; open full SKILL.md and references only on a match.
- If two skills overlap, pick one primary and at most one complementary.
- Never omit source code needed to verify a risky change.

4. Installation order and gates

#	Action	Gate before proceeding
1	Read-only inspection; Git status; package manager	User changes untouched; framework and lockfile identified
2	Dependency install (npm only if npm-managed)	No conflicting lockfile created
3	Verify UI library sources	Exact package, license, compatibility confirmed
4	Install the ten integrations one by one	Content and scripts inspected; no guessed command

#	Action	Gate before proceeding
5	Configure rules and registry	Skill discovery verified in current Antigravity IDE
6	Evaluate and install ONE graph engine	MCP connected; indexed; sample query passes
7	Evaluate Onyx separately	Deploy only after approval and resource/security review
8	Create docs and workflow	Docs match actual installed state
9	Lint, type-check, tests, build	Real results recorded
10	Final audit and report	Statuses clearly separated

5. Report statuses

Status	Meaning
INSTALLED AND VERIFIED	Installed, discovered by Antigravity, a relevant functional test passed
INSTALLED BUT NOT TESTED	Installed, no functional verification yet
CONFIGURED AND VERIFIED	Configuration active and tested
PARTIALLY CONFIGURED	Some parts work; remaining items listed
BLOCKED	Incompatibility, missing dependency, permission or source issue
PENDING USER INPUT	Exact repo, approval, credential or decision missing
NOT INSTALLED	Not installed; reason given

6. COPY-READY MASTER PROMPT

Copy everything between START and END into Antigravity. Fill in the two open-item URLs below first if you have them.

START MASTER PROMPT

ROLE

Act as a Principal Software Architect and AI Agent Infrastructure Engineer. Configure this existing project into a permanent, modular, token-efficient, graph-powered Antigravity engineering environment. This is a setup task. Do not redesign or rewrite the application.

OPEN INPUTS (fill in or leave blank)

- Animaster official source: _____ (blank = PENDING USER INPUT, do not guess)
- Task-chunking repository URL: _____ (blank = PENDING USER INPUT, use the local workflow in Phase 8)

SAFETY RULES

- Preserve all existing work. Start read-only. Record `git status --short` and the current branch.
- Compare and back up before overwriting any existing configuration.
- Never invent repository URLs, skill names, install commands, MCP tools, endpoints, package names or test results.
- Inspect third-party scripts and package lifecycle hooks before running them. Never run a remote curl-to-shell installer without review and explicit approval.
- Never commit secrets, tokens, .env files or personal data.
- Stop and ask before destructive actions, production deployment, schema migration, credential changes or scope expansion.
- Claim success only for what was actually verified.

PHASE 1 - INSPECT

- Identify framework, language, runtime, package manager, lockfile, build/test/lint commands.
- Review package.json, architecture, components, hooks, utilities, design tokens, routes, APIs, data access.
- Check existing `.agents/`, `.agent/`, AGENTS.md, MCP config and installed skills. Identify dependency overlaps.

PHASE 2 - DEPENDENCIES

- If npm-managed (package.json plus package-lock.json), run `npm install` at the root, then `npm ls --depth=0`.
- If pnpm, Yarn or Bun is in use, use its matching command and do not create a package-lock.json. Never delete or regenerate lockfiles to force npm. No unrelated upgrades.

PHASE 3 - UI LIBRARIES

- Animaster, Skiper UI (skiper-ui.com/docs/quick-start), Vengence UI (vengenceui.com).
- For each: verify official identity, registry, license, framework/Tailwind/shadcn compatibility and current install steps. Animaster stays unverified unless a source is given above.
- Install only components required by an approved task. No whole-library installs. No duplicate animation or icon packages without a documented reason.
- Create `docs/engineering/UI_LIBRARY_GUIDE.md` (version, source, license, installed components, usage).

PHASE 4 - TEN INTEGRATIONS

For each, verify the live repo: default branch, README, exact skill folder and name, SKILL.md, scripts, references, license, maintenance state, dependencies, install command, Antigravity compatibility. Official source only. Preserve upstream files and relative paths. No placeholder skills.

- 1 REA - github.com/morluto/rea - authorized reverse engineering.
- 2 Ponytail - github.com/DietrichGebert/ponytail - minimal, clean implementation.
- 3 Hallmark - github.com/Nutlope/hallmark - UI/design; verify identity first.
- 4 Humanizer - github.com/hannsxpeter/humanizer - language polish; never alter technical identifiers.
- 5 Onyx - github.com/onyx-dot-app/onyx - a full platform, handled in Phase 5. No fake SKILL.md.
- 6 Impeccable - github.com/pbakaus/impeccable - UI audit and refinement.
- 7 UI UX Pro Max - github.com/nextlevelbuilder/ui-ux-pro-max-skill - verify official CLI and generated skill path.
- 8 Last30Days - github.com/mvanhorn/last30days-skill - verify skill, sources, authentication.
- 9 Scrapling - github.com/D4Vinci/Scrapling - separate the Python toolkit from any Agent Skill; install only needed parts.
- 10 Diagram Design - github.com/cathrynlavery/diagram-design - verify skill and output formats.

If a URL, skill name or install command is invalid or ambiguous: stop that item, mark BLOCKED or PENDING USER INPUT, never substitute.

PHASE 5 - ONYX (OPTIONAL PLATFORM)

- Docs: github.com/onyx-dot-app/onyx/tree/main/deployment/docker_compose and .../backend/onyx/mcp_server
- Assess Docker/WSL2, CPU/RAM/disk, ports, volumes, persistence, security. Review any install script before approval to run it.
- Verify the MCP endpoint and auth from the deployed version. Use scoped, read-only tokens. Keep tokens out of project files. Index no secrets.
- Keep Onyx deferred until deployment is justified and approved. Create `docs/engineering/ONYX_INTEGRATION.md`.

PHASE 6 - CODE KNOWLEDGE GRAPH

- Evaluate CodeGraph, GitNexus and Aider RepoMap: real repo, maintenance, license and commercial limits, language support, Windows, Antigravity MCP, indexing, token overhead.
- Select ONE primary engine. Keep a short fallback note. No overlapping indexes or MCP servers without a measured need.
- Model where supported: files, symbols, imports/exports, callers/callees, React components and hooks, routes to services to data access, DB references, tests to targets, change impact.
- Before edits: search symbols, get compact context, check callers/callees, run impact analysis, identify tests, then read the actual source.
- After edits: re-index changed files, run tests, confirm index freshness.
- Store data in `.agent-data/code-graph/` (or engine default), add to .gitignore, one index per repo, never index secrets. Create `docs/engineering/CODE_GRAPH_GUIDE.md`.

PHASE 7 - ENGINEERING RULES

- No emoji or Unicode characters as UI icons. No hand-written inline SVG for ordinary icons. One approved icon library (Lucide React if compatible).
- No duplicate components: search code and graph for an existing component, hook or utility first. Reuse or extend. Abstract only for real reuse or clarity.
- Apply SOLID where fitting, DRY, KISS, YAGNI, separation of concerns, composition. Smallest complete solution; keep validation, security, error handling, accessibility, testability.
- No new dependencies without clear need. No API/DB contract changes without impact analysis and approval. Keep brand and design tokens. Target WCAG 2.2 AA.
- Write all of this into ONE file: `docs/engineering/ENGINEERING_RULES.md`, with sections for Icons, Component Reuse and Dependency Policy.

PHASE 8 - TASK CHUNKING (LOCAL WORKFLOW)

- For every substantial task: understand constraints and acceptance criteria; inspect architecture and Git; load only relevant skills; query graph/docs; write a short plan; split into independently verifiable chunks.
- Each chunk records goal, affected areas, dependencies, acceptance criteria, verification command. Implement one chunk at a time; run focused tests after risky chunks; review the full diff.
- Never mark a chunk done without evidence. Do not repeat finished work without new evidence. Create `docs/engineering/IMPLEMENTATION_WORKFLOW.md`.

PHASE 9 - ANTIGRAVITY CONFIGURATION

- Confirm supported paths in current official docs (antigravity.google/docs/skills, /customization-discovery, /cli/plugins). Do not assume CLI paths apply to the IDE.
- Create or update: AGENTS.md (short entry point), `.agents/skills/<name>/SKILL.md` (verified native skills only), MCP config where supported, hooks only if genuinely needed.
- Docs in `docs/engineering/`: README.md (index and setup status), ARCHITECTURE.md (verified only), SKILL_REGISTRY.md (source, exact name, license, install method, status, date), DECISIONS.md (dated), TESTING_AND_VERIFICATION.md (real commands).
- Do not copy upstream skill docs into AGENTS.md. Preserve upstream skill files.

PHASE 10 - ROUTING

REA (authorized only) for reverse engineering. Ponytail + graph for refactor and minimal implementation. UI UX Pro Max and/or Hallmark for new UI; Impeccable for audit. Humanizer for copy. Last30Days for recent research. Scrapling for authorized extraction. Diagram Design for diagrams. Onyx MCP only if deployed and connected; otherwise official Onyx docs. Never load every skill; open full instructions only when the task matches.

PHASE 11 - SECURITY AND MAINTENANCE

- Check scripts, lifecycle hooks, network access and license for each item. Never assume commercial-use permission.
- Scraping: respect terms, robots, rate limits, access controls. Reverse engineering: authorized targets only.

- Least privilege for MCP and services. No credentials in code, docs, graph or prompts. Back up before destructive operations. Do not silently upgrade skills; inspect upstream changes and re-test.

PHASE 12 - FINAL VERIFICATION AND REPORT

- Run only commands this project supports: dependency check, lint, type-check, tests, build, graph sample query and freshness, MCP tool test, skill discovery in Antigravity, git diff, secret scan. Do not claim unrun checks.
- Report: (1) framework, version, package manager; (2) Git status before/after; (3) each integration with exact source, license, and classification as skill, CLI/tool or platform; (4) status and functional test result per item; (5) UI libraries and installed components; (6) graph engine, index path, MCP status; (7) Onyx status or reason deferred; (8) rule and skill discovery status; (9) lint/type/test/build results; (10) blocked and pending items including Animaster and task-chunking if still open; (11) files changed and next steps.
- Statuses: INSTALLED AND VERIFIED; INSTALLED BUT NOT TESTED; CONFIGURED AND VERIFIED; PARTIALLY CONFIGURED; BLOCKED; PENDING USER INPUT; NOT INSTALLED.

FINAL REQUIREMENT

Build a maintainable, secure, token-efficient environment. Start with inspection and proceed phase by phase. Do not redesign the existing application.

END MASTER PROMPT

7. Reference links

- Antigravity Agent Skills: antigravity.google/docs/skills
- Antigravity customization discovery: antigravity.google/docs/customization-discovery
- Antigravity plugins and MCP: antigravity.google/docs/cli/plugins/
- Onyx repo, Docker Compose guide and MCP server README under github.com/onyx-dot-app/onyx
- Skiper UI quick start: skiper-ui.com/docs/quick-start
- Vengence UI: vengenceui.com

The agent must re-check live upstream instructions at execution time.